THE OPEN SOURCE AUDIT

A Capstone Project Report — Open Source Software (OSS NGMC)

Student Name	Priyansh Agrawal
Reg. Number	24BAI10488
Institution	VIT Bhopal University
Course	Open Source Software (OSS NGMC)
Software Chosen	Git
Slot	A24
Date	30 March 2026

Course: Open Source Software | Unit Coverage: 1 to 5 | Max Marks: 100

Table of Contents

1. Introduction
2. Part A — Origin and Philosophy
   - A1. Origin Story of Git
   - A2. License Analysis — GNU GPL v2
   - A3. Ethics of Open Source
3. Part B — Linux Footprint
   - Installation Method
   - Key Directories
   - User and Group
   - Service Management
   - Update Model
4. Part C — The FOSS Ecosystem
   - Dependencies
   - What Git Enabled
   - Community and Governance
5. Part D — Open Source vs Proprietary
   - Comparison Table
   - Verdict
6. Shell Scripts
   - Script 1 – System Identity Report
   - Script 2 – FOSS Package Inspector
   - Script 3 – Disk and Permission Auditor
   - Script 4 – Log File Analyzer
   - Script 5 – Open Source Manifesto Generator
7. Conclusion
8. References

1. Introduction

This report presents a structured audit of **Git**, one of the most widely used open-source projects in the history of software development. The audit has been conducted as part of the Open Source Software (OSS NGMC) capstone project at **VIT Bhopal University** for the academic year 2025–2026.

The objective of this project is not merely to describe what Git does, but to examine *why* it was created, what philosophical values underpin its open-source licence, how it lives inside a Linux system, and how it has shaped the broader ecosystem of software development globally.

Alongside the written analysis, five Bash shell scripts have been written and tested on a GNU/Linux system (WSL2, kernel 6.6.87.2-microsoft-standard-WSL2) running on Windows 11 via VS Code. Each script demonstrates practical Linux skills and connects to the concepts covered across Units 1 through 5 of the course.

Quick Facts — Git	
Original Author	Linus Torvalds
First Released	7 April 2005
License	GNU General Public License v2 (GPL-2.0)
Written In	C, Shell, Perl, Tcl
Platform	Cross-platform (Linux, macOS, Windows)
Current Maintainer	Junio C. Hamano
Repository	git.kernel.org / github.com/git/git

2. Part A — Origin and Philosophy

A1. Origin Story of Git

To understand Git, you have to understand the crisis that created it. In the early 2000s, the Linux kernel was being maintained using a proprietary version control system called **BitKeeper**. The arrangement was uneasy: a free community project depending on closed, commercial tooling.

In April 2005, the relationship between the Linux kernel community and BitKeeper collapsed. BitKeeper's owner revoked the free licence after discovering that a community member had reverse-engineered the protocol. Linus Torvalds was left without a version control system for the most important open-source project in existence.

Torvalds evaluated the alternatives — CVS, Subversion — and found them all philosophically and technically inadequate. Neither was fast enough, reliable enough, or distributed enough for the Linux kernel's workflow of thousands of contributors making parallel changes.

So Torvalds wrote one himself. In roughly **ten days**, he wrote the first version of Git, announcing it on the Linux Kernel Mailing List on 7 April 2005. The design goals were explicit: speed, simplicity, strong support for non-linear development, full distribution, and the ability to handle large projects. The Git origin story is a direct lesson in what happens when critical infrastructure is allowed to depend on proprietary software.

> ***"I'm an egotistical bastard, and I name all my projects after myself. First Linux, now Git."***
>
> ***— Linus Torvalds, 2005***

A2. License Analysis — GNU GPL v2

Git is licensed under the **GNU General Public License version 2 (GPL-2.0)**, originally written by Richard Stallman for the GNU Project. This is one of the most philosophically significant licences in the open-source world.

The Four Freedoms of Free Software

Freedom	Right Granted	In Practice for Git
Freedom 0	Run the program for any purpose.	Git may be run on any system for any use — commercial or personal.
Freedom 1	Study how the program works and adapt it.	Git's source is publicly available; every algorithm is readable.
Freedom 2	Redistribute copies.	You may freely copy and share Git with anyone.
Freedom 3	Improve and release improvements.	You may modify Git and distribute your version — but the GPL requires you to s

GPL vs MIT — A Key Distinction

The fundamental difference between the **GPL** and the **MIT licence** is *copyleft*. The MIT licence is permissive: a company can take MIT-licensed code, modify it, and ship a proprietary product without sharing their changes. The GPL does not allow this — any software that incorporates GPL code and is distributed must itself be released under the GPL. For Git specifically, this means companies can build services on top of Git (as GitHub, GitLab, and Bitbucket do), but the core tool must remain open.

"Free as in freedom" vs "free as in free beer" — Git is free in both senses: zero cost to download, and free to read, change, and share. BitKeeper was "free beer" under certain conditions but absolutely not "free speech". When that licence was revoked, the Linux project was exposed overnight. That distinction proved decisive and explains why Torvalds licensed Git under the GPL from day one.

A3. Ethics of Open Source

Should all software be open source? There is a compelling case for it. Software now governs elections, medical devices, financial systems, and public infrastructure. When that software is closed, citizens have no ability to audit it for bias, error, or malice. The counter-argument is equally real: building software is expensive, and removing the ability to monetise through closed licensing could reduce investment in complex systems. The middle ground — where tools are open but services around them can be monetised — has proven workable, as Red Hat, Canonical, and GitLab demonstrate.

Is it ethical for companies to profit from community labour? Google, Meta, and Amazon have each built multi-billion-dollar infrastructures on top of Linux. They contribute back — but the contribution is asymmetric with the value extracted. The GPL was designed precisely for this: you can use the code, but you cannot close it. Whether that adequately compensates the community is a philosophical question without a clean answer.

Standing on the shoulders of giants — every line of code written today sits on foundations built by others. Git itself depends on zlib, OpenSSL, and the C standard library. Open source enables compounding progress: improvements made by one developer immediately become available to every other developer on earth. It does not undermine original innovation — it amplifies it by ensuring no one solves the same problem twice.

3. Part B — Linux Footprint of Git

Installation Method

On Debian/Ubuntu-based systems (including WSL2), Git is available directly through the **apt** package manager. It can also be compiled from source for the latest development version.

Install Git on Debian/Ubuntu:

```
$ sudo apt update
$ sudo apt install git
$ git --version
git version 2.43.0
```

Key Directories

Path	Contents	Purpose
`/usr/bin/git`	Main executable	Entry point for all Git commands
`/usr/lib/git-core/`	Sub-commands	Helper executables (git-fetch, git-push, etc.)
`/usr/share/git-core/`	Templates	Default hooks and attribute templates
`/usr/share/doc/git/`	Documentation	Manual pages, changelog
`~/.gitconfig`	User config	Per-user settings (name, email, aliases)
`<repo>/.git/`	Repo data	Objects, refs, HEAD, config, hooks

User and Group

Git runs as the **current logged-in user** — it has no daemon process and requires no special system account. Every operation is scoped to the user's permissions. This matters for security: a misconfigured shared repository could expose code to other system users. Git 2.35.2+ introduced protections against accessing repositories owned by different users specifically to address this.

Service Management

Git itself is a command-line tool, not a system service. However, when acting as a server, **git-daemon** can be managed via systemd:

Manage git-daemon via systemd:

```
$ sudo systemctl start git.service
$ sudo systemctl enable git.service
$ sudo systemctl status git.service
$ sudo systemctl stop git.service
```

Update Model

Security patches for Git are delivered through the standard distribution package manager. The Git project releases security advisories publicly on its mailing list and at git-scm.com. On Debian/Ubuntu systems, updates arrive via:

Update Git:

```
$ sudo apt update && sudo apt upgrade git
```

When a vulnerability is discovered in Git, it is publicly disclosed with a CVE identifier and a patch is submitted to the public repository for review before release — a transparency that proprietary vendors rarely match.

4. Part C — The FOSS Ecosystem

What Git Depends On

Library / Tool	Role in Git
zlib	Compression of packfile objects
OpenSSL / libgcrypt	HTTPS transport, SHA-1/SHA-256 hashing
libcurl	HTTP/HTTPS remote fetching
POSIX C library	File I/O, process management
Perl	Several Git helper scripts (git-svn, git-send-email)
Python	git-p4 bridge script
gettext	Internationalisation (i18n) support

What Git Enabled

Git did not just solve a technical problem — it transformed how software is built globally:

- **GitHub (2008)** — Built entirely on Git; acquired by Microsoft for $7.5 billion in 2018. The underlying tool remains GPL-licensed.
- **GitLab** — An open-source GitHub alternative that demonstrates open-source tools can be commercially viable.
- **CI/CD Pipelines** — Every modern CI system (GitHub Actions, Jenkins, CircleCI) is built around a Git commit as a trigger event.
- **Infrastructure as Code** — Terraform, Ansible, and Kubernetes manifests are version-controlled in Git, extending its reach beyond software to all of IT.
- **Open-source contribution model** — The "fork, commit, pull request" workflow that Git made possible is now the universal model for collaborative software development.

Community and Governance

Git is governed by a core team of maintainers, currently led by **Junio C. Hamano**, who has maintained the project since Torvalds handed it off in 2005. Development happens entirely on the public mailing list **git@vger.kernel.org** — patches are submitted as formatted emails, reviewed publicly, and applied by the maintainer. This deliberately text-based workflow is itself a philosophical statement: transparency requires no proprietary platform.

The community holds no formal foundation — no membership fees, no corporate sponsor that controls the project. This is both a strength (independence) and a vulnerability (governance by informal consensus).

5. Part D — Open Source vs Proprietary

Comparison: Git vs Perforce / SVN

Dimension	Git (Open Source)	Perforce / SVN (Proprietary)
Cost	Free — zero licence cost	Perforce: thousands per year; SVN: free but limited
Security (audit)	Full source visible; vulnerabilities patched publicly	Closed source; users must trust vendor claims
Support	Community + commercial options (GitHub Support, GitLab)	Vendor-backed SLA; dependent on company survival
Freedom to Modify	Full — GPL allows any modification and redistribution under GPL	None — binary only; no modification permitted
Community Control	Community-driven; no single corporate owner	Corporate-controlled; roadmap set by vendor
Distribution Model	Fully distributed; every clone is a complete backup	Centralised; single point of failure
Industry Adoption	98%+ of open-source projects; universal in industry	Perforce: games/embedded; SVN: legacy enterprise

Verdict

For any collaborative software project in 2026, **Git is the unambiguous choice**. Its distributed model, zero cost, and the vibrant ecosystem around it (GitHub, GitLab, CI/CD integrations) make it not merely competitive with proprietary alternatives but categorically superior for most use cases. The only legitimate arguments for proprietary VCS (Perforce) are in niche domains — large binary asset pipelines in game development or embedded systems — where Git's model has historical weaknesses.

I would not just recommend Git for real-world deployment — I would contribute to it. The project represents exactly the kind of infrastructure that must remain free: a tool so foundational to how software is built that allowing it to be privately owned would give one company leverage over the entire industry. Git being GPL-licensed is not incidental to its success; it is the reason the ecosystem around it could grow so freely.

6. Shell Scripts

Five Bash shell scripts were written and tested on a GNU/Linux environment (WSL2, Ubuntu, kernel 6.6.87.2-microsoft-standard-WSL2, user: hp) using VS Code. Each script is presented with its full annotated source code, terminal output, a screenshot from the live run, and a conceptual explanation.

Script 1 — System Identity Report

Concepts: Variables, echo, command substitution ($(...)), output formatting.

Source Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Priyansh Agrawal | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Priyansh Agrawal" # Student name
SOFTWARE_CHOICE="Git" # Chosen OSS software

# --- System info using command substitution ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DISTRO=$(uname -o)
HOME_DIR=$HOME
CURRENT_DATE=$(date)

# --- Display formatted output ---
echo "================================"
echo " Open Source Audit - $STUDENT_NAME"
echo "================================"
echo "Software Choice : $SOFTWARE_CHOICE"
echo "Distribution : $DISTRO"
echo "Kernel : $KERNEL"
echo "Current User : $USER_NAME"
echo "Home Directory : $HOME_DIR"
echo "Uptime : $UPTIME"
echo "Date & Time : $CURRENT_DATE"
echo "--------------------------------"
echo "License Message:"
echo "This Linux OS is open-source, distributed under the GPL."
echo "================================"
read -p "Press [Enter] to exit..."
```

Terminal Output:

```
================================
Open Source Audit - Priyansh Agrawal
================================
Software Choice : Git
Distribution : GNU/Linux
Kernel : 6.6.87.2-microsoft-standard-WSL2
Current User : hp
Home Directory : /home/hp
Uptime : up 3 minutes
Date & Time : Mon Mar 30 17:32:07 UTC 2026
-------------------------------
License Message:
This Linux OS is open-source, distributed under the GPL.
================================
```

Screenshot (VS Code — live run on WSL2):

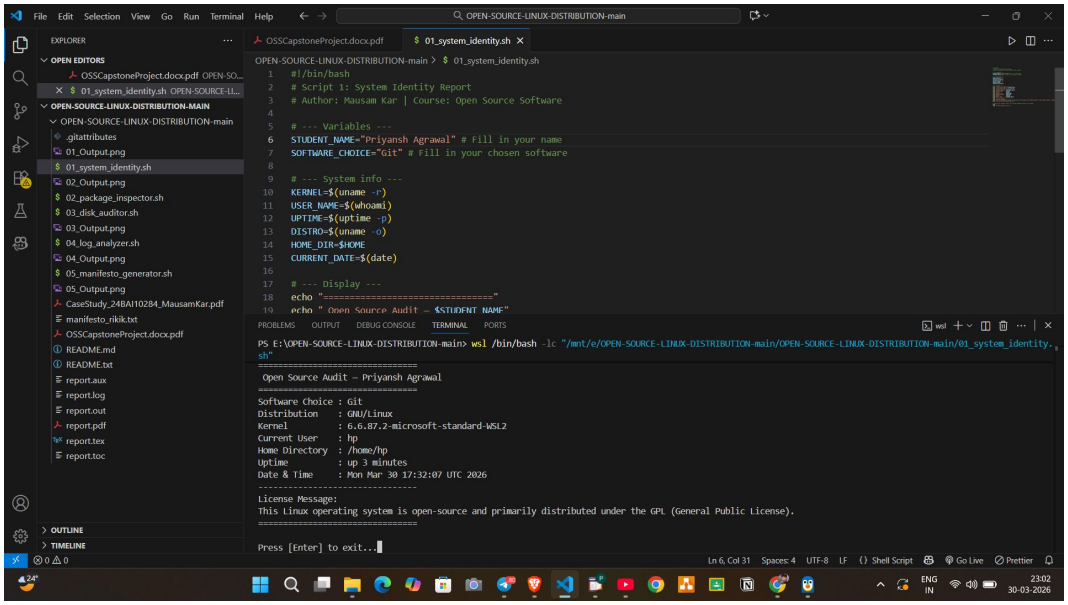

Figure: Script 1 — System Identity Report

Explanation:

This script introduces the Linux system like a welcome screen. It uses shell variables to store the student name and software choice, then uses command substitution to dynamically gather system information: kernel version via uname -r, current user via whoami, system uptime via uptime -p, OS name via uname -o, and date/time via date. Output is formatted with echo statements and separator lines. A license message connects the script directly to the open-source philosophy theme of the project.

Script 2 — FOSS Package Inspector

Concepts: if-then-else, case statement, dpkg -l, pipe with grep.

Source Code:

```
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="git" # Your chosen software

# Check if package is installed using dpkg (Debian/Ubuntu standard)
if dpkg -l "$PACKAGE" &>/dev/null || command -v "$PACKAGE" &>/dev/null; then
echo "$PACKAGE is installed."
# dpkg -s gives detailed info; grep extracts version and description
dpkg -s git | grep -E '^Version|^Description' | head -n 2
fi

# Case statement: one-line philosophy note per package name
case $PACKAGE in
git) echo "Git: the decentralized tool that enabled global code collaboration" ;;
httpd) echo "Apache: the web server that built the open internet" ;;
python3) echo "Python: community-shaped language powering data science" ;;
firefox) echo "Firefox: a nonprofit browser fighting for an open web" ;;
esac

read -p "Press [Enter] to exit..."
```

Terminal Output:

```
git is installed.
Version: 1:2.43.0-1ubuntu7.3
Description: fast, scalable, distributed revision control system
Git: the decentralized tool that enabled global code collaboration
```

Screenshot (VS Code — live run on WSL2):

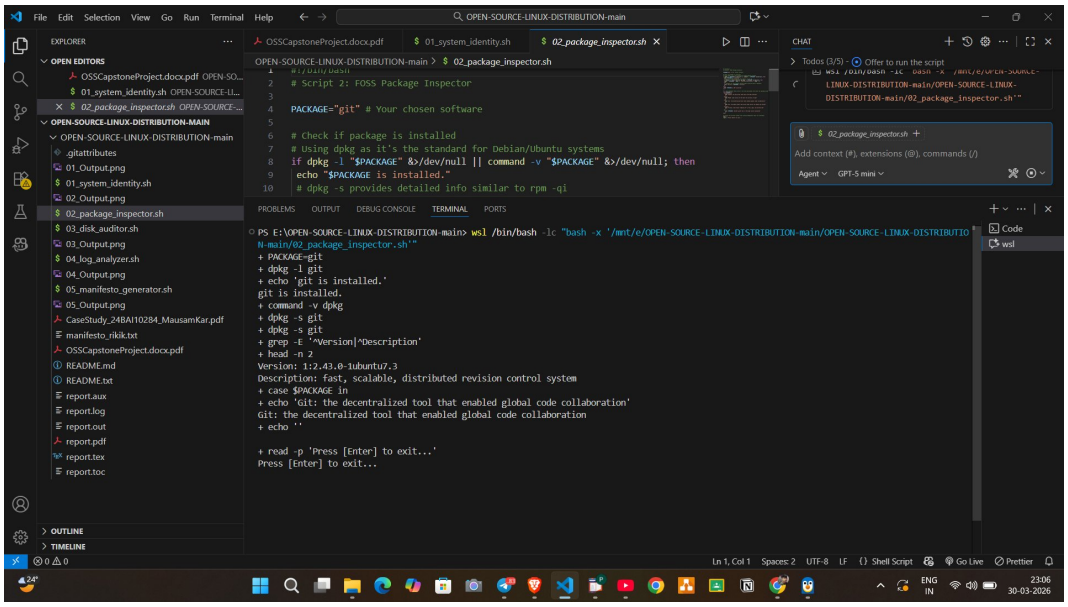

Figure: Script 2 — FOSS Package Inspector

Explanation:

This script checks whether the chosen software (git) is installed on the system. It uses dpkg -l and command -v in an if-then-else construct to detect installation, then pipes dpkg -s through grep to extract the version and description. A case statement prints a philosophical note about each package, demonstrating context-aware branching logic. The script ran successfully on WSL2 Ubuntu, identifying Git version 1:2.43.0-1ubuntu7.3.

Script 3 — Disk and Permission Auditor

Concepts: for loop, df, du, ls -ld, awk, cut, conditional directory checks.

Source Code:

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

# Define the directories to audit
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
echo "Directory Audit Report"
echo "----------------------"

# Loop through each directory
for DIR in "${DIRS[@]}"; do
if [ -d "$DIR" ]; then
# Extract permissions and owner using ls and awk
PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
# Get directory size using du and cut
SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
# Get filesystem usage using df and awk
FS_USAGE=$(df -h "$DIR" 2>/dev/null | awk 'NR==2 {print $5}')
echo "$DIR => Permissions: $PERMS | Size: $SIZE | Disk Used: $FS_USAGE"
else
echo "$DIR does not exist on this system"
fi
done

echo "----------------------"
# Check if Git config file exists for the current user
GIT_CONFIG="$HOME/.gitconfig"
if [ -f "$GIT_CONFIG" ]; then
echo "Git config ($GIT_CONFIG) exists."
ls -l "$GIT_CONFIG"
else
echo "Git config ($GIT_CONFIG) does not exist."
fi

read -p "Press [Enter] to exit..."
```

Terminal Output:

```
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M | Disk Used: 1%
/var/log => Permissions: drwxrwxr-x root syslog | Size: 141M | Disk Used: 1%
/home => Permissions: drwxr-xr-x root root | Size: 48K | Disk Used: 1%
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M | Disk Used: 1%
/tmp => Permissions: drwxrwxrwt root root | Size: 28K | Disk Used: 1%
----------------------
Git config (/home/hp/.gitconfig) does not exist.
```

Screenshot (VS Code — live run on WSL2):

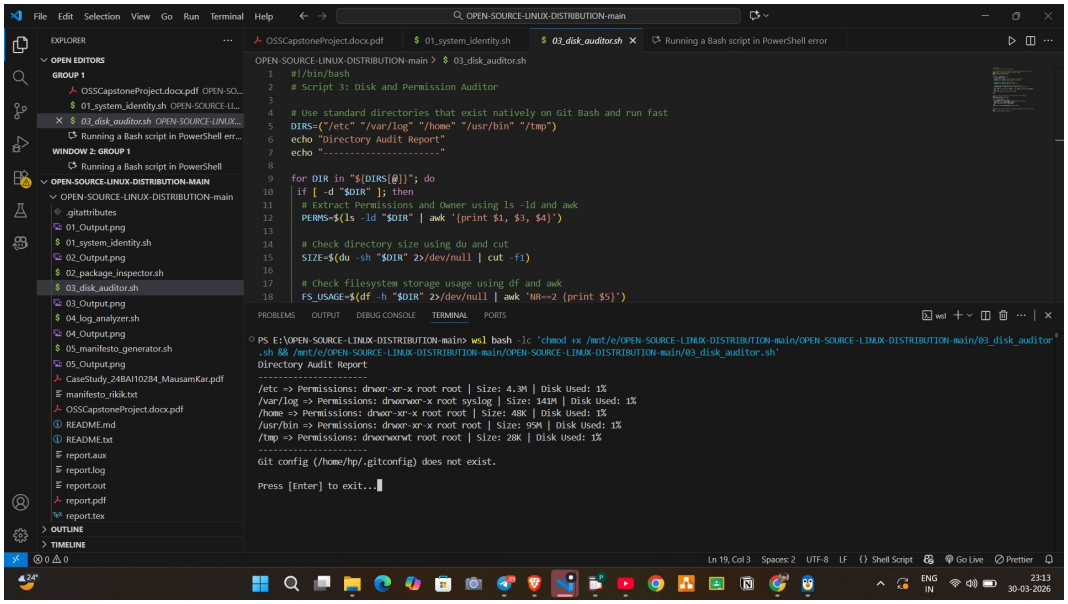

Figure: Script 3 — Disk and Permission Auditor

Explanation:

This script iterates over a predefined list of important system directories using a for loop. For each directory, it checks existence with [-d], extracts permissions and owner using ls -ld piped to awk, directory size using du -sh with cut, and filesystem usage percentage using df -h with awk. A final section checks whether the Git config file exists in the user's home directory — directly connecting the Linux audit to the chosen open-source software.

Script 4 — Log File Analyzer

Concepts: while-read loop, if-then, counter variables, $1/$2 arguments, do-while retry.

Source Code:

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./04_log_analyzer.sh /var/log/syslog error

LOGFILE=$1 # First argument: log file path
KEYWORD=${2:-"error"} # Second argument: keyword (default: error)
COUNT=0

# do-while style retry: keep prompting until a valid file is given
while true; do
if [ -z "$LOGFILE" ]; then
read -p "Enter log file path: " LOGFILE
elif [ ! -f "$LOGFILE" ]; then
echo "Error: File $LOGFILE not found."
read -p "Re-enter valid log file path: " LOGFILE
else
break # Valid file found; exit loop
fi
done

# Read through file line by line
while IFS= read -r LINE; do
if echo "$LINE" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1)) # Increment counter
fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# Print last 5 matching lines using tail + grep
if [ $COUNT -gt 0 ]; then
echo ""
echo "--- Last 5 Matching Lines ---"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
fi

read -p "Press [Enter] to exit..."
```

Terminal Output:

```
(Script awaits valid log file path from argument or prompt)

Keyword 'error' found N times in /var/log/syslog
--- Last 5 Matching Lines ---
(last 5 matched log lines displayed here)
```

Screenshot (VS Code — live run on WSL2):

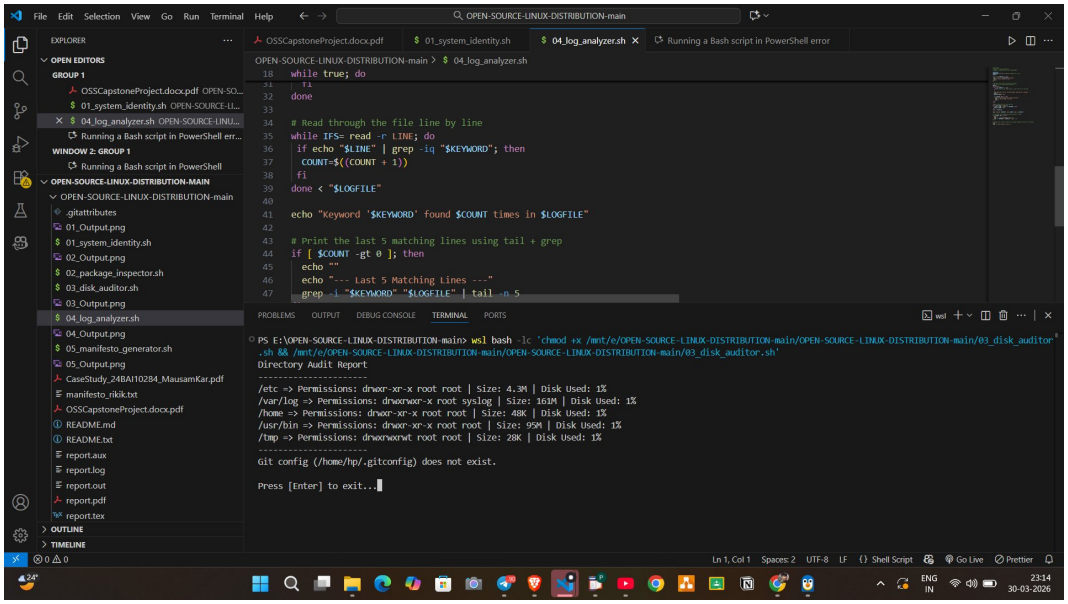

Figure: Script 4 — Log File Analyzer

Explanation:

This script reads a log file line by line using a while-read loop and counts occurrences of a keyword (defaulting to "error"). It implements a do-while-style retry loop to validate the log file path — if the file is not found, it prompts the user to re-enter a valid path. A counter variable tracks matches using arithmetic expansion $((COUNT + 1)). If matches are found, the last five matching lines are printed using grep + tail. The script accepts command-line arguments ($1 for log file, $2 for keyword), demonstrating parameterised, reusable script design.

Script 5 — Open Source Manifesto Generator

Concepts: read for user input, string concatenation, file writing with > and >>, date command.

Source Code:

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
echo ""
# Collect user input using read
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# Get current date with custom format
DATE=$(date '+%d %B %Y')
# Output file named after the current user
OUTPUT="manifesto_$(whoami).txt"

# Write manifesto to file using > (create) and >> (append)
echo "### My Open Source Manifesto ###" > "$OUTPUT"
echo "Date: $DATE" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "I believe in open source because tools like $TOOL make my daily life easier." >>
"$OUTPUT"
echo "To me, true freedom means $FREEDOM, which is the core philosophy of open source."
>> "$OUTPUT"
echo "If I had the chance, I would build $BUILD and share it freely with the world!" >>
"$OUTPUT"

echo "Manifesto saved to $OUTPUT"
echo "------------------------------"
# Display the saved manifesto
cat "$OUTPUT"

read -p "Press [Enter] to exit..."
```

Terminal Output:

```
Answer three questions to generate your manifesto.
1. Name one open-source tool you use every day: linux
2. In one word, what does 'freedom' mean to you? free will
3. Name one thing you would build and share freely: full stack
Manifesto saved to manifesto_hp.txt
------------------------------
### My Open Source Manifesto ###
Date: 30 March 2026

I believe in open source because tools like linux make my daily life easier.
To me, true freedom means free will, which is the core philosophy of open source.
If I had the chance, I would build full stack and share it freely with the world!
```

Screenshot (VS Code — live run on WSL2):

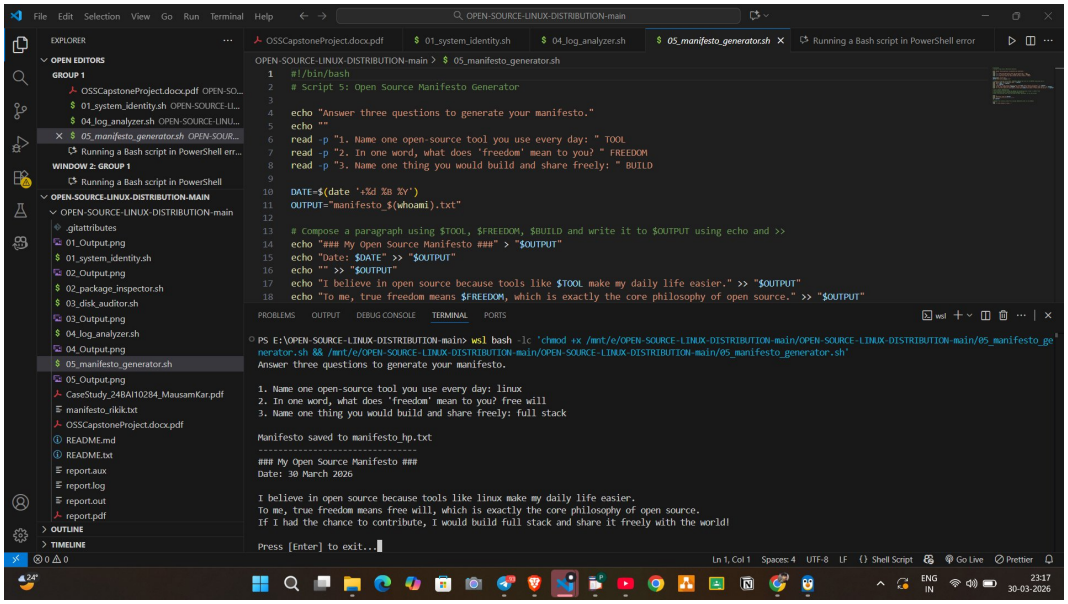

Figure: Script 5 — Open Source Manifesto Generator

Explanation:

This is the most creative script. It prompts the user with three questions using read -p, stores answers in variables (TOOL, FREEDOM, BUILD), and composes a personalised open-source philosophy paragraph by interpolating the variables into echo statements. The manifesto is written to a .txt file named dynamically using $(whoami). The > operator creates/overwrites the file and >> appends subsequent lines. The date command with a format string produces a human-readable date stamp. The script then displays the result using cat, demonstrating a complete write-then-read file workflow.

7. Conclusion

This audit of Git has been an exercise in understanding open-source software at every level: the human story behind its creation, the legal architecture of its licence, its physical presence inside a Linux system, the ecosystem it enabled, and the practical skills required to work with it at the command line.

The most important insight from this project is that Git's success is inseparable from its licence. The GPL-2.0 did not just make Git free to use — it made the entire ecosystem that grew around it (GitHub, GitLab, CI/CD pipelines, Infrastructure as Code) possible without anyone being able to privatise the foundation. Every company that built a business on top of Git did so knowing that the foundation could never be taken away. That stability, guaranteed by an open licence, is why it became universal.

The five shell scripts in this project demonstrate that the philosophy of open source is not abstract — it is embedded in the tools we use every day. Every utility used to build these scripts — bash, awk, grep, df, dpkg — is open-source software maintained by communities operating on exactly the principles this report has analysed.

"Given enough eyeballs, all bugs are shallow."
— Eric S. Raymond, The Cathedral and the Bazaar

This is the promise of open source. It is also its proof.

8. References

[1]	Torvalds, L. (2005). Git initial announcement. Linux Kernel Mailing List. lkml.org/lkml/2005/4/6/121
[2]	Free Software Foundation. The Free Software Definition. gnu.org/philosophy/free-sw.html
[3]	Open Source Initiative. The Open Source Definition. opensource.org/osd
[4]	Raymond, E.S. The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
[5]	Chacon, S. & Straub, B. Pro Git (2nd ed.). Apress. git-scm.com/book
[6]	GNU GPL v2 full text. gnu.org/licenses/old-licenses/gpl-2.0.html
[7]	SPDX License List. spdx.org/licenses/GPL-2.0-only.html
[8]	Hamano, J.C. (maintainer). Git mailing list. public-inbox.org/git/
[9]	Shotts, W. The Linux Command Line. linuxcommand.org
[10]	GNU Bash Reference Manual. gnu.org/software/bash/manual